

Given an array price[] of length N, representing the prices of the stocks on different days the task is to find the maximum profit possible for buying and selling the stocks on different days using transactions where any number of transactions are allowed.

Input Format:

Line-1:A single integer `n` representing the number of days.

Line-2:`n` space-separated integers representing the stock prices for each day.

Output Format:

Print an integer representing the maximum profit possible. If no profit is possible, print `0`.

Example-1

Input:

6

7 1 5 3 6 4

output =

7

Explanation:

Purchase on 2nd day. Price = 1.

Sell on 3rd day. Price = 5.

Therefore, profit = $5 - 1 = 4$.

Purchase on 4th day. Price = 3.

Sell on 5th day. Price = 6.

Therefore, profit = $4 + (6 - 3) = 7$.

Example-2

Input

5

1 2 3 4 5

output:

4

Explanation:

Purchase on 1st day. Price = 1.

Sell on 5th day. Price = 5.

Therefore, profit = $5 - 1 = 4$.

1. Single Transaction (Buy once, sell once)

The algorithm tracks the minimum buying price and updates the maximum profit if selling at the current price yields a higher return (6:06-6:48).

cpp

// Logic pseudo-code

```
int minprice = INTMAX;
```

```
int max_profit = 0;
```

```
for (int price : prices) {
```

```
    if (price < min_price)
```

```
        min_price = price;
```

```
    else if (price - minprice > maxprofit)
```

```
        maxprofit = price - minprice;
```

```
}
```

2. Infinite Transactions (Multiple buy/sell)

This approach adds the profit whenever the next day's price is higher than the current day's price, effectively capturing every upward trend (16:38-17:33).

cpp

// Logic pseudo-code

```
int max_profit = 0;
```

```
for (int i = 0; i < n - 1; i++) {
```

```
    if (prices[i] < prices[i + 1]) {
```

```
        max_profit += (prices[i + 1] - prices[i]);
```

```
    }
```

```
}-
```